A SS-CNN on an FPGA for Handwritten Digit Recognition

Jiong Si
Department of Electrical and Computer Engineering
University of Nevada, Las Vegas
Las Vegas, NV, U.S.A.
sij1@unlv.nevada.edu

Evangelos Yfantis
Department of Computer Science
University of Nevada, Las Vegas
Las Vegas, NV, U.S.A.
yfantis@cs.unlv.edu

Sarah L. Harris
Department of Electrical and Computer Engineering
University of Nevada, Las Vegas
Las Vegas, NV, U.S.A.
sarah.harris@unlv.edu

***Abstract*—This paper describes a Super-Skinny Convolutional Neural Network (SS-CNN) and its implementation on a Cyclone IVE field programmable gate array (FPGA), for handwritten digit recognition. This SS-CNN performs state-of-the-art recognition accuracy but with fewer layers and less neurons. Using parameters with 8 bits of precision, the FPGA solutions of this SS-CNN show no recognition accuracy loss when compared to the 32-bit floating point software solution. In addition to high recognition accuracy, both of the proposed FPGA solutions are low power and require little FPGA area. The proposed hardware solutions indicate a 67 to 355 times power savings potential when compared to the software solution. Thus, our SS-CNN provides a high-performance, low computation demands, hardware friendly, and power efficient solution.**

Keywords—SS-CNN, FPGA, MNIST, deep learning, machine learning, hardware acceleration

I. INTRODUCTION

In recent years, with the support of increasingly powerful computation resources, neural networks have become larger and more powerful. They continue to reach higher recognition accuracies, sometimes even surpassing human ability. However, such deep neural networks have many layers and millions of parameters. For example, AlexNet [1] has 8 layers and 62 million parameters, VGG Net [2] has 16 layers and 138 million parameters, and GoogLeNet [3] has 22 layers and 7 million parameters. Several techniques have been introduced to decrease the number of neural network parameters including such methods as neural network connection pruning [4] which deletes the "unimportant" connections between layers, sparse neural network [5] which deletes the "unimportant" neurons in the network, and feature channel pruning [6] which deletes the "unimportant" feature channels in the network. These methods have produced parameter size compression of the neural networks without decreasing recognition accuracy.

When compared to recent networks recognizing the MNIST dataset [7], for example the LeNet-5 neural network [8], the convolutional neural network (CNN) we designed is shallower, with fewer neurons and parameters. Perhaps an even more compelling feature of the proposed CNN is that it does not use FPGA-unfriendly techniques such as normalization. Thus, it can be readily mapped onto an FPGA.

In this paper, we introduce the Super-Skinny Convolutional Neural Network (SS-CNN) that we designed. Beyond input and output layers, it has only three additional layers: one convolutional layer, one Max-Pooling layer, and one fully connected layer. It has only 39,541 total parameters, which requires 0.04 MB of memory after we quantized the parameters to 8-bit precision in the FPGA implementation. Despite its minimal design and FPGA area, it has high recognition accuracy on handwritten digits in the MNIST dataset. The inference portion of the SS-CNN fits easily on the low-cost Cyclone IV FPGA platform. Because of the FPGA implementation's low power, low computation demands, and low memory requirements, this system can be readily integrated into various portable devices.

The remaining sections of this paper describe our software and hardware designs of the SS-CNN used to recognize the handwritten digits in MNIST dataset. Section II first introduces the overall algorithm and design process and then describes the activation function approximation, system controller, and the computation units used in each layer of the FPGA system. Section III describes the power consumption of the system when compared to the software implementation. This section also describes two hardware solutions on an FPGA, one with sequential calculations for the six feature channels and one with these calculations in parallel. Finally, Section IV summarizes our findings and conclusions.

II. METHODS

This section describes the Super-Skinny CNN (SS-CNN) algorithm, the CNN network architecture, MNIST dataset, and the sigmoid activation function. It also describes the CNN hardware design on the FPGA including the overall system architecture, control, and computation units.

A. MNIST Dataset

We use the MNIST dataset of handwritten digits to train and test the SS-CNN network. The MNIST dataset includes a training set of 60,000 images and their labels, and a testing set of 10,000 images and their labels. Fig. 1 shows five

978-1-7281-3885-5/19/$31.00 ©2019 IEEE

Authorized licensed use limited to: Auckland University of Technology. Downloaded on June 06,2020 at 18:07:06 UTC from IEEE Xplore. Restrictions apply.

image examples from the MNIST dataset with their labels shown above each image. Each image is 28×28 pixels (784 total pixels), as shown by the x and y labels on the images in Fig. 1.

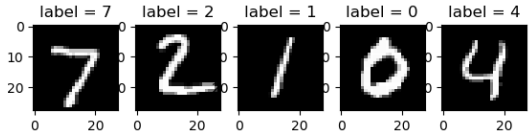


Fig. 1. Examples of digits in the MNIST dataset

B. *Super-Skinny Convolutional Neural Network (SS-CNN)*

We designed, built, and tested a novel CNN architecture, the SS-CNN, for recognizing the MNIST dataset. This network results in state-of-the-art performance in terms of accuracy, circuit area, and power. Designing such a CNN is not a random task, but requires exploratory analysis using mathematical, statistical and image processing knowledge. The autocorrelation function of neighboring pixels is a function of the distance between the two pixels and attenuates as the distance increases. We explored convolutional kernels ranging from 5×5 to 28×28 data points, and we found that performance (prediction accuracy) remained the same for convolutions with 5×5 to 8×8 pixel kernels. However, for convolutions using more than 8×8 or lower than 5×5 pixel kernels, the performance decreased. Thus, we chose convolutions using 5×5 pixel kernel. After computing the principal components of 5×5 segments of a large group of images, using stride one, we concluded that the six principal components account for almost 100% of the total variance. For this reason we chose six feature maps at the convolutional layer. While we also explored using seven and eight feature maps, we found no difference in prediction accuracy when compared to using six feature maps. For the Max-Pooling layer we used a weight of one and an offset of zero because we found no increase in prediction accuracy when we used the weights and offsets computed during the learning stage. Following the Max-Pooling layer we introduced a fully connected layer. The number of neurodes in the fully connected layer was determined in a stepwise approach. We began with 35 neurodes and gradually increased the neurodes to explore the effect on performance. The accuracy prediction reached its maximum at 45 neurodes; using more than 45 neurodes did not improve the performance. Thus we chose 45 neurodes for the hidden layer. During training, after 12 epochs, the probability of correct classification for the training data was 99.92% and that of the test data was 98.68%. At the end of epoch 15 the probability of correct classification for the training data was 99.987% and that of the test data was 98.76%. As the number of epochs increased, the probability of correct classification for the training data approached one and that of the test data was 99.9%. A feedforward final test program with a GUI interface was written that implemented the architecture described above using the coefficients obtained after epoch 15. Fig. 2 shows a snapshot of the program running.

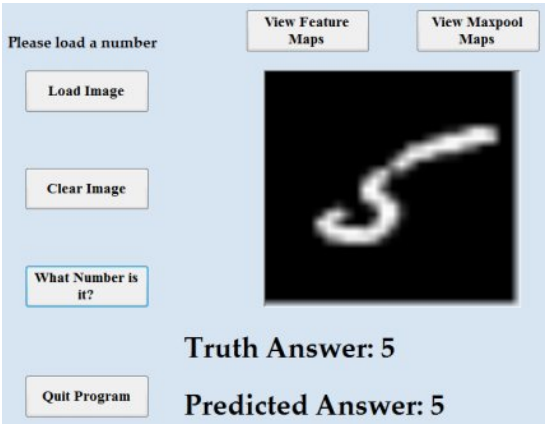


Fig. 2. Executable action of test program

Fig.3 shows the overall architecture of the SS-CNN. The network processes 28×28 pixel grayscale images of handwritten digits (see Fig. 1) from which we derive six convolutional feature maps. To derive a feature map, we consider a 5×5 pixel segment of the input image, with stride one and offset zero. Thus, for one convolutional feature map we need 26 (i.e., 5×5+1) parameters and for all six feature maps we need 156 (i.e., 26×6). Each of the six feature maps has 576 (i.e., 24×24) components. Each feature map also derives a 12×12 Max-Pooling layer by taking the maximum of a 2×2 non-overlapping segment in the feature map. Going from a feature map to the pooling layer the weight is fixed to one, and the offset is zero. All of the 864 neurodes of the Max-Pooling layer are connected to the 45 neurodes of the fully connected layer plus one offset, for a total of (864+1)×45=38,925 coefficients. Each of the 10 output layers is fully connected with the 45 neurodes plus one offset for a total of (45+1)×10=460 coefficients. The total number of coefficients is 156+38925+460=39,541 coefficients.

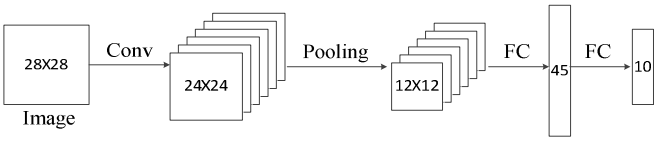


Fig. 3. Super-skinny convolutional neural network

All weights are initialized with uniformly distributed independent random variables in the interval [-0.5, 0.5]. The neural network is trained with 60,000 training images, and enough epochs needed to maximize the probability of correct classification without overfitting. The output is sensitive to the choice of learning rate and the activation function. The training probability of correct classification was 99.75% while the probability of correct classification of the test data was 98.95%, which is comparable to neural network architectures with many more layers that require more time for training and testing than our architecture. The architecture introduced in this paper is highly parallelizable using multithreading programming in software and parallel computation units in hardware. The weights obtained after training the CNN were used by a feedforward CNN having the architecture described above and shown in Fig. 3. Fig. 2 shows the GUI of the software version of the feedforward neural network used to classify the 10,000 MNIST test images.

Authorized licensed use limited to: Auckland University of Technology. Downloaded on June 06,2020 at 18:07:06 UTC from IEEE Xplore. Restrictions apply.

C. Activation Function

While several activation functions exist, including the ReLU and sigmoid activation functions, we chose to use the sigmoid activation function in our SS-CNN because it results in the best recognition accuracy. Its equation and a graph of the sigmoid function are shown in equation (1) and Fig. 4.

$$f(x) = \frac{1}{1 + e^{-x}} \tag{1}$$

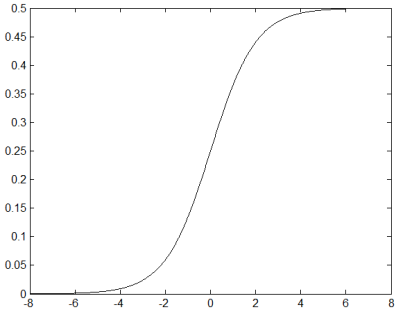


Fig. 4. Sigmoid function

Because the sigmoid function requires exponential and division operations, which are both costly in terms of hardware area, we instead use a piecewise linear approximation method [9] to approximate the original sigmoid function. As shown in Fig. 5, the piecewise linear function (represented by the blue line) closely approximates the original sigmoid function (represented by the red dotted line). The piecewise linear approximation offers hardware friendly calculations that can be readily implemented on an FPGA. As shown in Table I, the piecewise linear approximation replaces the exponent and division in the original sigmoid function with shifting and addition.

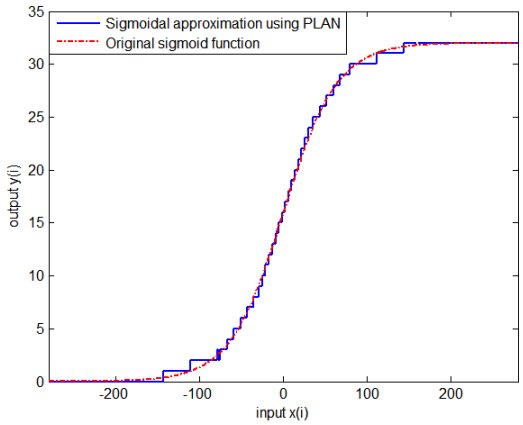


Fig. 5. Sigmoid function and its approximation using piecewise linear approximation of a nonlinear function (PLAN)

TABLE I. IMPLEMENTATION OF PLAN [9]

Output: y = F(x)	*Input Condition*
y = 32	x ≥ 160
y = (x << 5) [a] + 27	76 ≤ x < 160
y = (x << 3) [b] + 20	32 ≤ x < 76
y = (x << 2) [c] + 16	0 ≤ x < 32
y = 1 – y	x < 0

[a] right shift 5 bits [b] right shift 3 bits [c] right shift 2 bits

D. FPGA System

This section describes the CNN network we built in SystemVerilog on a Cyclone IVE FPGA. The system diagram of the SS-CNN network is shown in Fig. 6 and is described in detail in this section. We implement the network using parameters with 8-bit precision for all inputs, outputs, and trained weights.

The CNN hardware system (see Fig. 6) consists of external memories, on-chip registers, and calculation units on an FPGA. The external memories, the *Image/Label RAM* and *Weights RAM*, store the image/label data and trained weights. The on-chip registers, the *Conv Result Reg*, store the calculation results of the convolutional layer. The system controller, the *Controller*, manages the dataflow of the whole system. The computation units on the FPGA system, *Convolutional Layer*, *Pooling Layer*, and *FC layers*, perform the calculations of the neural network. The final result is displayed on the *Display* unit.

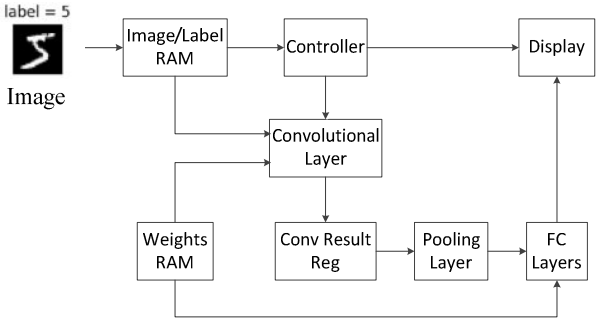


Fig. 6. FPGA system architecture

C.1 System Controller

The *Controller* module manages the dataflow during forward propagation. Its main module is the finite state machine (FSM) defined by the state transition diagram in Fig. 7. In the *Idle* state, the FSM waits for the *Start_conv* signal to assert in order to start the convolution operation. After the image data is read from external memory, the *Start_conv* signal is asserted, which moves the FSM to the forward (*Fwd*) state and triggers the forward propagation process. After forward propagation computations are complete, the FSM displays the image label and predicted result on the 7-segment displays and then returns to the *Idle* state to continue processing the next image.

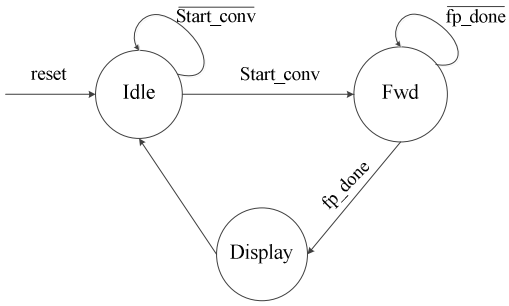


Fig. 7. Controller finite state machine (FSM)

C.2 Convolutional Layer

The *Convolutional Layer* unit performs feature extraction from the input data images by performing convolution operations using six 5×5 kernel frames. As shown in Fig. 8, one 5×5 pixel kernel frame convolves with 5×5 pixels of image data to generate one pixel in the convolutional layer. The convolution kernel has six frames, and the stride of

Authorized licensed use limited to: Auckland University of Technology. Downloaded on June 06,2020 at 18:07:06 UTC from IEEE Xplore. Restrictions apply.

each frame is one. No padding is added to the original images, so we get six 24×24 frames in the convolutional layer as shown in Fig. 3 and Fig. 8.

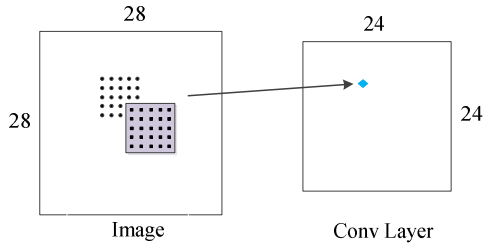


Fig. 8. Convolution for one kernel frame

The basic calculation cell in the convolutional layer is the 5×5 multiplication net, as shown in Fig. 9. Each clock cycle the 5×5 multiplication net calculates the multiplication of 5×5 pixels from the input image and one convolution kernel frame (which consists of a 5×5 block of data).

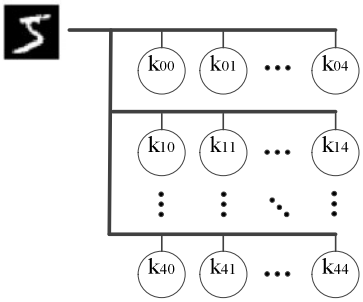


Fig. 9. Multiplication of image data and one kernel frame

To complete the convolution operation, we accumulate the 25 (i.e., 5×5) multiplication results. We then use an adder tree to add them together as shown in Fig. 10. The variables M_{00} … M_{04}, M_{10} … M_{14}, M_{20} … M_{24}, M_{30} … M_{34}, M_{40} … M_{44} represent the 25 multiplication results from the 5×5 multiplication net as shown in Fig. 9. The accumulation operation requires five clock cycles.

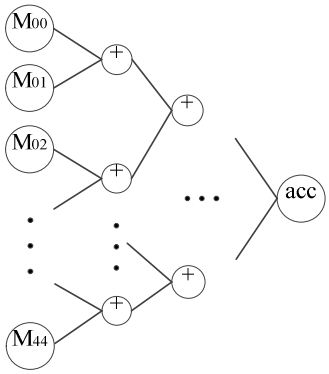


Fig. 10. Adder tree for image and kernel frame multiplication results

The convolution kernel consists of six distinct 5×5 convolution frames as discussed in section II.B. We designed two separate FPGA solutions; one configuration processes the six feature channels sequentially, and the other processes them in parallel.

Fig. 11 shows the calculation steps of the convolutional layer when the feature channels run sequentially. After completion of the *Kernel Multiply* and *Tree Acc* operations just described above, the system adds a *Bias* to the accumulated data and then passes the data through the sigmoid activation function. At this point the system goes to the *Byte Ready* state which indicates that one convolution data value in the convolutional layer has finished calculating. After completing this computation, the FSM goes back to *Idle* state to continue calculating the next data value in the convolutional layer. We repeat this process 3,456 (i.e., 6×24×24) times to process all of the convolution operations in one image.

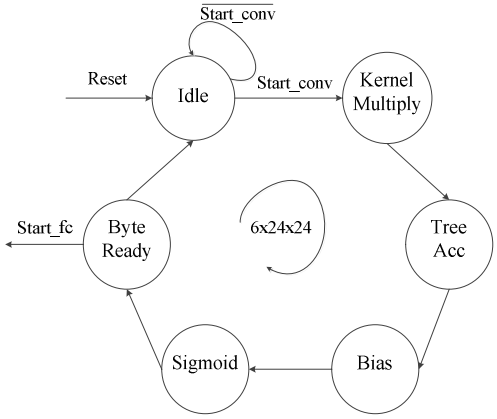


Fig. 11. Convolutional layer finite state machine (FSM) with sequential feature channel

When the six feature channels are processed in parallel, we replicate the convolutional layer hardware on the FPGA six times. As expected, as shown in Fig. 12, the FSM of the convolutional layer is the same but each of the six parallel FSMs only repeat 576 (i.e., 24×24) times to finish the convolution operations for one image.

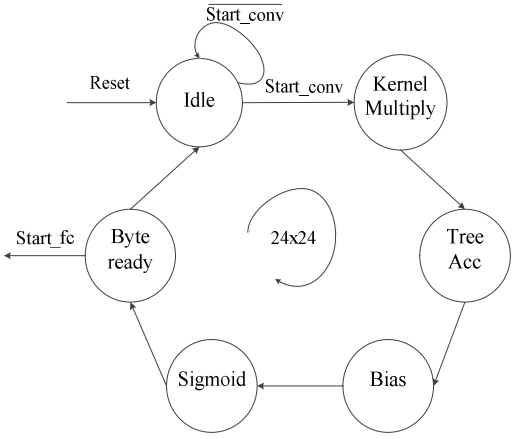


Fig. 12. Convolutional layer finite state machine (FSM) with parallel feature channel

After the calculations of the convolutional layer are complete, the Max-Pooling layer is processed in the following clock cycle, as shown in Fig. 13. The Max-Pooling layer picks the maximum value in non-overlapping 2×2 boxes in the six 24×24 frames in the convolutional layer. Thus, the Max-Pooling layer results in six 12×12 frames, as shown in Fig. 3 and Fig. 13. At the same time, the *Start_fc* signal in Fig. 11 and Fig. 12 is asserted to start the calculations in the proceeding two fully connected layers.

Authorized licensed use limited to: Auckland University of Technology. Downloaded on June 06,2020 at 18:07:06 UTC from IEEE Xplore. Restrictions apply.

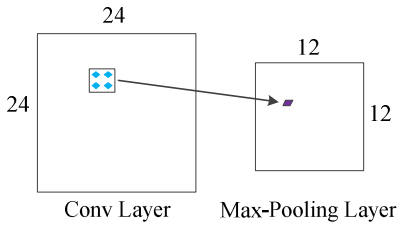


Fig. 13. Max-Pooling layer

C.3 Fully Connected Layers

Our proposed neural network, the SS-CNN, has one fully connected hidden layer and one fully connected output layer. The inputs of the fully connected hidden layer are the six 12×12 frames from the Max-Pooling layer. The two operations of multiplication and accumulation run sequentially for each neuron in the hidden layer. Thus, calculating the result in one neuron of the fully connected hidden layer requires 864 (i.e., 6×12×12) clock cycles, which is the size of its input data – or, in other words, the size of the Max-Pooling layer output. The results of the 45 neurons in the fully connected hidden layer are calculated in parallel, so the calculation of the fully connected hidden layer costs 864 cycles plus the time to add a bias and to pass the data through the sigmoid function.

The input of the fully connected output layer is the 45 neuron outputs from the hidden layer. The output layer performs the same calculations on its inputs as the hidden layer. Thus, the calculation of the fully connected output layer requires 45 clock cycles plus the time to add a bias and to pass the data through the sigmoid function.

Fig. 14 shows the calculation process of the two fully connected layers. After receiving the starting signal *Start_fc* from the convolutional layer FSM in Fig. 11 or Fig. 12, the calculations of the fully connected hidden layer are triggered. After 864 (i.e., 6×12×12) operations of multiplication and addition, the FSM moves to the calculation of the fully connected output layer. Again, after repeating 45 times, the forward propagation calculations are complete. The *FP_done* signal is then asserted, which informs the system controller in Fig. 7 that the system is ready to process the next image, and it also tells the system to display the prediction result of the current image.

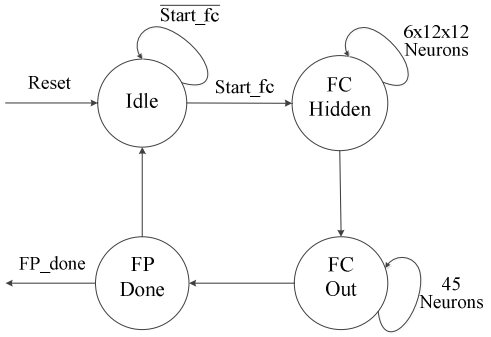


Fig. 14. Fully connected layers finite state machine (FSM)

III. Results and Discussion

The experimental results described in this section show that the proposed SS-CNN produces high recognition accuracy on handwritten digits. Its implementation on an FPGA result in lower execution time and lower clock frequency without loss in prediction accuracy, which also indicates potential power saving. Moreover, our two FPGA solutions can each fit on a low cost Cyclone IVE FPGA.

A. Recognition Accuracy

The recognition accuracy of the 8-bit hardware design is 98.8% as compared to the 32-bit floating point software implementation that achieves an accuracy of 99.0%. The FPGA hardware solutions exhibit negligible recognition accuracy loss while using only 25% of the bits used in the software solution.

As shown in Table II, when compared with the 2- and 3-layer fully connected neural networks (FC) we designed in [10][11], the recognition accuracies of handwritten digits using the SS-CNN increased by 3.5-8.7% in the software solutions and 3.2-8.8% in the hardware solutions.

TABLE II. Recognition Accuracy

Implementation	*Recognition Accuracy*
SS-CNN in Software	99.0%
SS-CNN in Hardware	98.8%
3-layer FC in Software	95.5%
3-layer FC in Hardware	95.6%
2-layer FC in Software	90.3%
2-layer FC in Hardware	90.0%

B. Performance

The performance differences between software and hardware solutions, as measured by clock frequency and execution time, are summarized in Table III. The clock frequency of the software solution is determined by the CPU frequency (3.6 GHz), and the clock frequencies of the FPGA designs are the highest frequencies at which the designs can run on the Cyclone IVE FPGA. The execution time is the inference time of the 10,000 test images in the MNIST dataset.

TABLE III. Execution Time

Solution	*Clock Frequency*	*Execution Time*	*Potential Power Savings*
Software	3.6 GHz	6.5 seconds	1×
Sequential channel on FPGA	37 MHz	9.4 seconds	67×
Parallel channel on FPGA	30 MHz	2.2 seconds	355×

As shown in Table III, the FPGA solution with sequential feature channel processing runs with a 37 MHz clock frequency, which is 97 times lower than the software solution. Even though its execution time is longer, it still has 67 (i.e., (3.6GHz/37MHz)×(6.5s/9.4s)) times the potential power savings when compared to the software solution. The potential power savings of the parallel feature channel processing solution is much higher. This solution has both a lower clock frequency and a lower execution time, which results in 355 (i.e., (3.6GHz/30MHz)×(6.5s/2.2s)) times the potential power savings when compared to the software solution.

C. FPGA Area

Both the sequential and parallel FPGA solutions fit on Intel's low-cost Cyclone IVE FPGA. The SS-CNN hardware design with sequential feature channel processing requires

Authorized licensed use limited to: Auckland University of Technology. Downloaded on June 06,2020 at 18:07:06 UTC from IEEE Xplore. Restrictions apply.

20k logic elements (LEs) on the Cyclone IVE FPGA, which is only 17% of the total LEs available on that chip. The parallel feature channel processing solution requires 98k logic elements, which is 86% of the total available LEs, as shown in Table IV. Because only the convolutional layer is parallelized – that is, the Max-Pooling and fully connected layers are the same in both hardware solutions – so the FPGA area requirement of the parallel feature channel processing solution is less than six times that of the sequential feature channel processing solution.

TABLE IV. FPGA AREA

FPGA Solution	*Logic elements (% use [d])*
Sequential channel	20k (17%)
Parallel channel	98k (86%)

[d] Intel's Cyclone IVE FPGA

IV. CONCLUSIONS

The CNN architecture presented here offers a high-performance, compact neural network compared with other recent neural networks applied to handwritten digit recognition.

The two FPGA solutions of the proposed SS-CNN also offer high-performance and low power alternatives to the software solution. The 8-bit hardware designs achieve recognition accuracy (~99%) that is similar to the 32-bit floating point software solution. Moreover, the two proposed hardware designs on the FPGA indicate 67 to 355 times the potential power savings when compared to the software solution.

REFERENCES

[1] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet Classification with Deep Convolutional Neural Networks," *NIPS*, pp. 1–9, 2012.

[2] K. Simonyan and A. Zisserman, "Very Deep Convolutional Networks for Large-Scale Image Recognition," *International Conference on Learning Representations (ICRL)*, pp. 1–14, 2015.

[3] C. Szegedy *et al.*, "Going Deeper with Convolutions," *CVPR2015*, 2014.

[4] S. Han, H. Mao, and W. J. Dally, "Deep Compression: Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding," *ICLR 2016*, pp. 1–14, 2015.

[5] J. Frankle and M. Carbin, "The Lottery Ticket Hypothesis: Finding Sparse, Trainable Neural Networks," Mar. 2018.

[6] X. Gao, Y. Zhao, Ł. Dudziak, R. Mullins, and C. Xu, "Dynamic Channel Pruning: Feature Boosting and Suppression," Oct. 2018.

[7] http://yann.lecun.com/exdb/mnist/, Last accessed November 1, 2019.

[8] P. H. Yann Lecun, Leon Bottou, Yoshua Bengio, "Gradient-Based Learning Applied to Document Recognition," 1998.

[9] H. Amin, K. M. Curtis, and B. R. Hayes-Gill, "Piecewise linear approximation applied to nonlinear function of a neural network," *Circuits, Devices and Systems, IEE Proceedings* -, vol. 144, no. 6, pp. 313–317, 1997.

[10] J. Si and S. L. Harris, "Handwritten Digit Recognition System on an FPGA," *CCWC 2018*, pp. 402–407, 2018.

[11] J. Si, S. L. Harris, and E. Yfantis, "A Dynamic ReLU on Neural Network," *2018 IEEE 13th Dallas Circuits and Systems Conference (DCAS)*, pp. 1–6, 2018.

Authorized licensed use limited to: Auckland University of Technology. Downloaded on June 06,2020 at 18:07:06 UTC from IEEE Xplore. Restrictions apply.